



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Architecture & Design Specification (SAD)

AttendEase – Student Attendance Management System Team 3 | Course UE24CS341A | PES University

Revision History

Version	Date	Description	Author(s)
1.0	03 October 2026	Initial SAD covering architecture and design for AttendEase.	Team 3
1.1	03 October 2026	Corrected FR-037/038 ownership, SEC/NFR traceability, security-architecture wording, API paths, and Figures 2-3 to match the SRS and the sequence diagrams.	Team 3

Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026

1. Introduction

1.1 Purpose

This Software Architecture & Design Specification (SAD) describes the system architecture and detailed design of AttendEase, a web-based Student Attendance Management System. It translates the functional and non-functional requirements captured in the AttendEase Software Requirements Specification (SRS) into an architectural structure, component design, and interaction design that the development team will implement. This document is intended to guide implementation, support code review, and serve as a reference for testing and maintenance.

1.2 Scope

This SAD covers the architecture of the AttendEase web application, including its layered component structure, the technology stack, the chosen architectural pattern, security architecture, and the detailed design of two representative workflows (attendance marking and correction-request approval) via UML sequence diagrams. It also documents the REST API design, error-handling strategy, and traceability between architectural components and the requirements defined in the SRS.

1.3 Intended Audience

- Team 3 developers implementing AttendEase modules.
- Course instructor and evaluators assessing the Level 3 mini-project.
- Future maintainers extending or reviewing the system.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
SAD	Software Architecture & Design Specification
SRS	Software Requirements Specification
DRF	Django REST Framework
ORM	Object-Relational Mapping
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — a threat-modelling framework
JWT	JSON Web Token, used for stateless session authentication
RBAC	Role-Based Access Control

2. Document Overview

2.1 How to Use This Document

Section 3 (Architecture) should be read first by anyone evaluating the overall system structure, technology choices, and security posture. Section 4 (Design) should be read by developers implementing or reviewing specific modules, as it drills down into sequence-level interactions and the API contract. The Appendices provide supporting glossary, references, and tooling information.

2.2 Related Documents

- AttendEase Software Requirements Specification (SRS), v1.2, Team 3.
- AttendEase Project Synopsis, Team 3.

- AttendEase Software Test Plan (STP), Team 3.
- PES University SE Level 3 Mini-Project Guidelines and Deliverables Checklist.

3. Architecture

3.1 Goals and Constraints

Architectural goals:

- Clear separation of concerns between presentation, API, business logic, and data access, so that each of the four team-owned roles can develop and test their module independently.
- Maintainability and readability, given the project's academic timeline and four-person team.
- Security by design for authentication, role-based access, and sensitive attendance data.
- Testability — each service is unit-testable in isolation from the HTTP layer.

Constraints:

- Must be implemented in Python/Django within a single academic semester by a 4-member team.
- Must use Git/GitHub with a CI/CD pipeline (GitHub Actions) and Jira/GitHub Projects for sprint tracking, per course guidelines.
- Must run on commodity/free-tier hosting suitable for a student project (e.g., containerized deployment via Docker).

3.2 Stakeholders and Concerns

Stakeholder	Primary Concern
Students	Fast, accurate view of personal attendance; easy correction requests.
Faculty	Quick, reliable attendance marking; simple approval workflow for corrections.
Administrators	Accurate academic setup (courses, sections, faculty mapping); reliable reports.
Development Team (Team 3)	A modular design that maps cleanly onto the 4 owned roles and allows parallel development.
Course Evaluators	Clear, traceable architecture and design that satisfies IEEE-style documentation expectations.

3.3 Component (UML) Diagram

The diagram below shows AttendEase's layered component architecture: the Presentation Layer (role-specific web UIs), the API/Controller Layer (Django REST Framework endpoints), the Business Logic Layer (services, grouped by the four team-owned roles), the Data Access Layer (repositories over the Django ORM), the PostgreSQL database, and the external SMTP email service used for correction-request notifications.

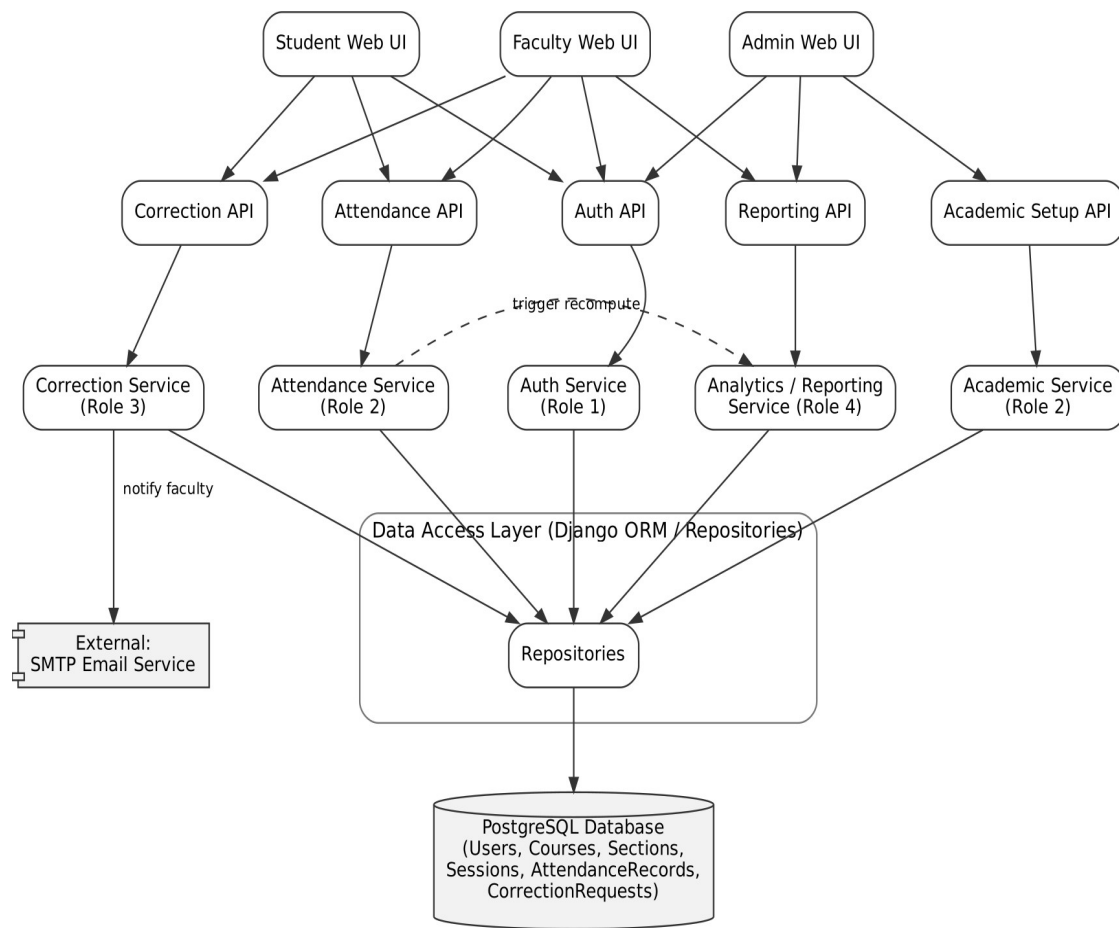


Figure 1: AttendEase component architecture (layered view).

3.4 Component Descriptions

Component	Responsibility
Student / Faculty / Admin Web UI	Role-specific browser-rendered views (Django templates / Bootstrap 5) that collect input and render data returned by the API layer.
Auth API & Auth Service	Registration, login, JWT/session issuance, and role-based access control (Role 1 — Atharva Ashish Vyas).
Academic Setup API & Academic Service	Course, section, and timetable management; faculty-to-section assignment (Role 2 — Arundhati K).
Attendance API & Attendance Service	Attendance session creation and per-student attendance recording (Role 2 — Arundhati K). Also enforces the duplicate-prevention and edit-window rules (FR-037, FR-038) at runtime, on behalf of Role 3's ownership of those requirements (see Correction API & Correction Service below).
Correction API & Correction Service	Raising, listing, and approving/rejecting attendance correction requests; owns the attendance-record duplicate-prevention and edit-window policy (FR-037, FR-038), which the Attendance Service enforces at runtime (Role 3 — Anshdeep Singh Sachdeva).
Reporting API & Analytics/Reporting Service	Attendance-shortage identification and report generation (Role 4 — Bhoomika Dayanand Jituri).

Component	Responsibility
Repositories (Data Access Layer)	Encapsulates all Django ORM queries, isolating business services from persistence details.
PostgreSQL Database	Durable storage for users, courses, sections, sessions, attendance records, and correction requests.
SMTP Email Service (external)	Sends correction-request notification emails to faculty.

3.5 Chosen Architecture Pattern and Rationale

AttendEase adopts a Layered (N-tier) architecture — Presentation, API/Controller, Business Logic (Service), and Data Access layers — implemented on top of Django and Django REST Framework.

Rationale:

- The layered pattern maps directly onto Django's own MVT conventions, minimizing framework friction for a student team.
- Each of the four team-owned roles (User Access & Student Portal; Academic & Attendance Setup; Attendance Updates & Corrections; Attendance Analytics & Reporting) corresponds to one vertical slice of API + Service, allowing the team to work largely in parallel with limited merge conflicts.
- A microservices architecture was considered and rejected: the added operational complexity (service discovery, inter-service networking, distributed transactions) is not justified for a single-database, single-deployment, academic-scale system with a 4-person team and a one-semester timeline.
- The Service layer isolates business rules (e.g., duplicate-attendance validation, shortage thresholds) from both the HTTP layer and the ORM, keeping logic unit-testable.

3.6 Technology Stack and Data Stores

Layer	Technology
Backend framework	Python 3.11, Django 4.x, Django REST Framework
Frontend	Django Templates, Bootstrap 5, vanilla JavaScript
Database	PostgreSQL 14+
Authentication	Django's auth system + JWT (djanoestframework-simplejwt) for API sessions
CI/CD	GitHub Actions (lint, test, build on every pull request) — chosen over Jenkins because it needs no self-hosted runner/infrastructure and integrates natively with the team's existing GitHub repository, which fits a 4-person student project better than operating a separate CI server.
Static Analysis	SonarQube (or an equivalent, e.g. SonarCloud's free tier) integrated into the GitHub Actions pipeline, to flag code smells, duplication, and common security issues on every pull request alongside the mandatory code review.
Containerization	Docker, docker-compose for local/staging environments
Project tracking	Jira / GitHub Projects (Kanban board, linked issues and PRs)
Email	SMTP (Django email backend) for correction-request notifications

3.7 Risks and Mitigations

Risk	Mitigation
Concurrent attendance edits by faculty causing data races	Database-level unique constraints on (session, student); service-layer validation before insert/update.
Scope creep given the 4-role split and semester timeline	Sprint-based plan of work with weekly Jira/GitHub Projects tracking and fixed sprint boundaries.
Team members unfamiliar with Django REST Framework	Shared code-review checklist and pairing during early sprints (Role: API integration & code review).
Loss of uncommitted work / merge conflicts	Mandatory feature-branch workflow with pull requests and CI checks before merge.
Sensitive attendance data exposure	Role-based access control and HTTPS-only deployment (see Section 3.9).

3.8 Traceability to Requirements

The table below maps architectural components to the functional, non-functional, and security requirements they implement, as defined in the AttendEase SRS Sections 5 and 6. FR-037 and FR-038 are listed under Correction API / Correction Service to match their ownership under Role 3 in the SRS (Section 5.3.1), even though the Attendance Service enforces them at runtime (see Section 3.4).

Component	SRS Requirement IDs
Auth API / Auth Service	FR-001–FR-005, FR-034; SEC-001, SEC-002
Academic Setup API / Academic Service	FR-006–FR-013
Attendance API / Attendance Service	FR-014–FR-019; NFR-001, NFR-002, NFR-004; SEC-004
Correction API / Correction Service	FR-024–FR-028, FR-037, FR-038; NFR-003; SEC-006
Reporting API / Analytics Service	FR-029–FR-033, FR-035, FR-036; NFR-001
Student/Faculty/Admin Web UI	FR-020–FR-023; NFR-007; and the presentation aspects of all FRs above

Cross-cutting requirements that apply across all components rather than a single one: NFR-005, NFR-006, and NFR-008 (maintainability, test coverage, and deployability apply system-wide); SEC-003 (server-side RBAC enforcement, applied per endpoint across every component) and SEC-005 (HTTPS, enforced at the infrastructure/reverse-proxy level).

3.9 Security Architecture

Security architecture is derived using a lightweight STRIDE threat-modelling pass over the system's trust boundaries (browser ↔ API, API ↔ database, API ↔ SMTP).

STRIDE Category	Representative Threat	Architectural Mitigation
Spoofing	An attacker impersonates a faculty or admin account.	Password hashing (PBKDF2/bcrypt via Django), JWT-based session tokens, mandatory login throttling/lockout after repeated failed attempts (SEC-002).
Tampering	A student modifies attendance data directly via API manipulation.	Server-side role-based authorization checks on every write endpoint; DB-level constraints.
Repudiation	A faculty member denies having approved/rejected a correction request.	Audit fields (acted_by, acted_at) stored on every CorrectionRequest; immutable status-change log.

STRIDE Category	Representative Threat	Architectural Mitigation
Information Disclosure	Attendance records of one section leak to unauthorized students.	Query-level scoping by authenticated user's role and enrolled sections; HTTPS in transit.
Denial of Service	Bulk/automated requests overwhelm the attendance-marking endpoint.	Rate limiting at the API gateway/reverse proxy layer; pagination on list endpoints.
Elevation of Privilege	A student account attempts to call admin-only academic-setup endpoints.	DRF permission classes enforcing RBAC per endpoint, verified in Section 6.3 (Security Objectives/Requirements) of the SRS.

4. Design

4.1 Design Overview

This section details two representative end-to-end workflows at the sequence level, the REST API contract, and cross-cutting design concerns (error handling, logging, UX). The two workflows were chosen because they represent the system's core write-paths and both faculty- and student-facing interactions: (1) Faculty starts a session and marks attendance, and (2) a student raises a correction request which a faculty member subsequently approves.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram 1 — Faculty Starts Session & Marks Attendance

Traces to FR-014, FR-015, and FR-016 (session creation, roster display, and per-student marking) and FR-037 (duplicate-record validation, shown as the "validate duplicates" step). Bulk-marking (FR-017), session closure (FR-018/FR-019), and the attendance-record edit-window flow (FR-038) are not depicted in this diagram; they follow the same Attendance Service but are separate flows described in SRS Sections 5.2.2 and 5.3.1.

Sequence Diagram 1: Faculty Starts Session & Marks Attendance

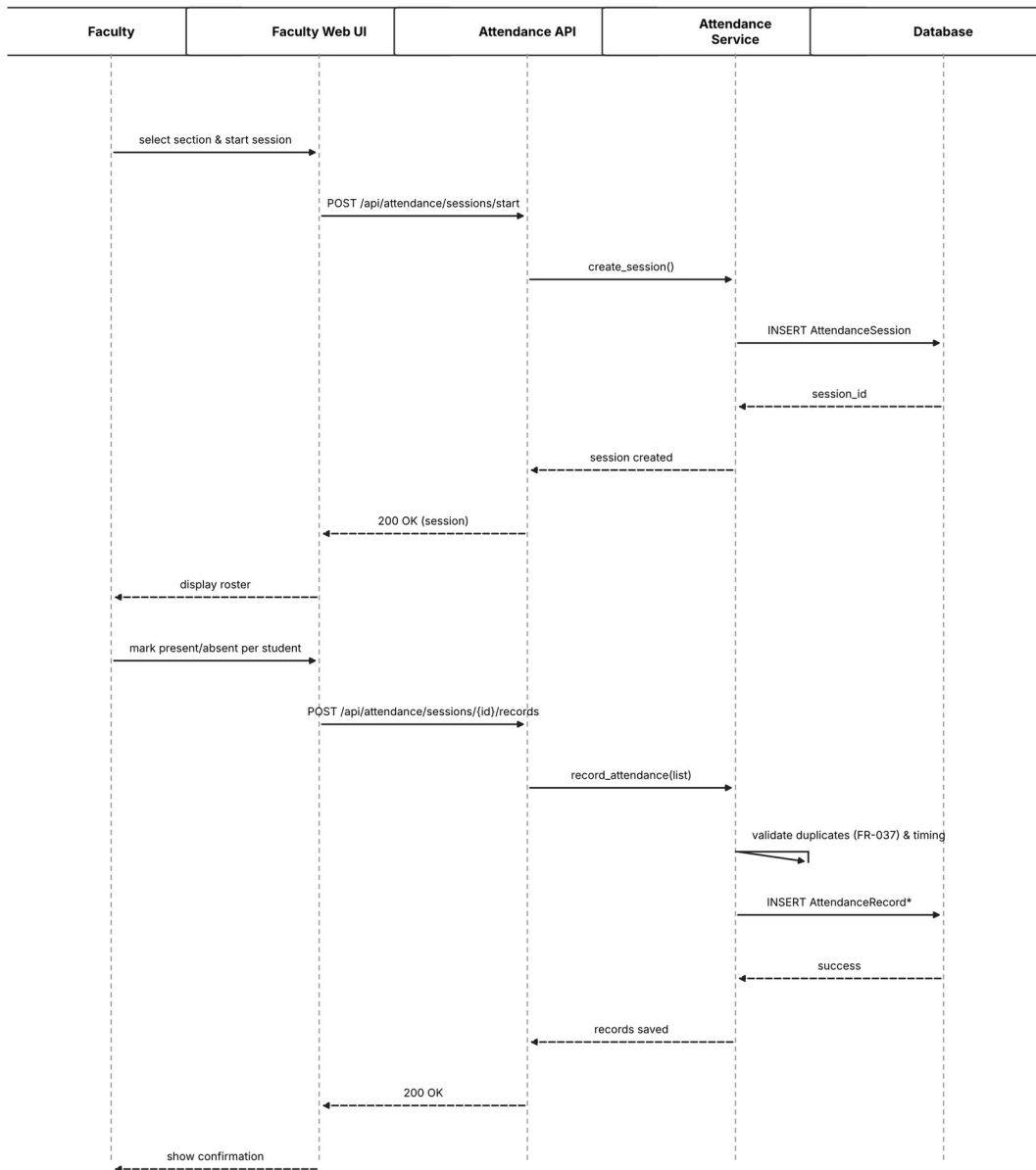


Figure 2: Faculty starts an attendance session and marks student attendance.

4.2.2 Sequence Diagram 2 — Student Raises Correction Request; Faculty Approves or Rejects

Traces to FR-024 through FR-028 (Attendance Correction Requests), including both the approval path (FR-027) and the rejection path (FR-028). The diagram separates the CorrectionRequest status update (with audit fields acted_by/acted_at, per SEC-006) from the downstream AttendanceRecord update, and shows the SMTP Service as an explicit external dependency rather than an internal step, consistent with the component diagram in Figure 1.

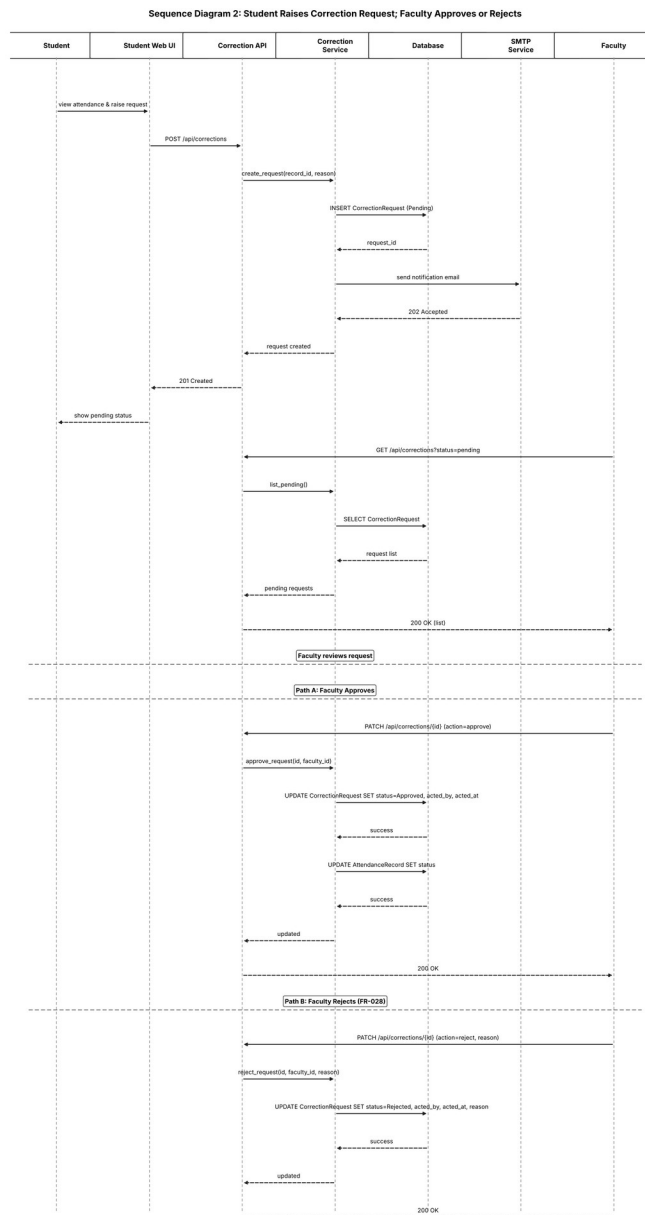


Figure 3: Student raises a correction request; faculty reviews and either approves or rejects it.

4.3 API Design

Representative REST endpoints exposed by the API/Controller layer (full contract to be maintained in the project's OpenAPI/Swagger specification during implementation). All endpoints other than registration and login require a valid JWT bearer token; the Auth/Role column states which role(s) may call the endpoint once authenticated.

Method & Path	Description	Auth / Role
POST /api/auth/register	Register a new user (student or faculty) with name, email, password, and role (FR-001).	Public
POST /api/auth/login	Authenticate and issue a JWT access/refresh token pair (FR-003).	Public
GET /api/academic/courses	List courses and sections.	Authenticated (any role)

Method & Path	Description	Auth / Role
POST /api/academic/assignments	Assign a faculty member to a course-section (FR-013).	Admin
POST /api/attendance/sessions/start	Start a new attendance session for a section (FR-014).	Faculty
POST /api/attendance/sessions/{id}/records	Submit attendance records for a session; rejects duplicates per FR-037.	Faculty
GET /api/attendance/dashboard	Retrieve the authenticated student's own attendance summary (FR-020).	Student
POST /api/corrections	Raise an attendance correction request against one of the student's own records (FR-024).	Student
PATCH /api/corrections/{id}	Approve (FR-027) or reject (FR-028) a correction request for a section the faculty member owns.	Faculty
GET /api/reports/defaulters	Generate an attendance-shortage/defaulters report (FR-031).	Faculty / Admin

Request/response contract, validation, and status codes — illustrated for the two endpoints exercised by Figures 2 and 3; the remaining endpoints follow the same conventions:

- POST /api/attendance/sessions/{id}/records — Request body: {"records": [{"student_id": int, "status": "present"|"absent"}]}. Validation: session must exist and be open; student_id must be enrolled in the session's section; no (session_id, student_id) pair may already have a record (FR-037). Responses: 200 OK with the saved records on success; 400 Bad Request for a malformed body; 403 Forbidden if the session does not belong to the calling faculty member; 404 Not Found for an unknown session; 409 Conflict for a duplicate record.
- PATCH /api/corrections/{id} — Request body: {"action": "approve"|"reject", "reason": string (required when action="reject")}. Validation: the request must currently be Pending; the calling faculty member must own the section the underlying AttendanceRecord belongs to; reason is required for a rejection (FR-028). Responses: 200 OK with the updated request on success (acted_by and acted_at are set server-side, never accepted from the client); 400 Bad Request if reason is missing on a rejection; 403 Forbidden if the faculty member does not own the section; 404 Not Found for an unknown request; 409 Conflict if the request is no longer Pending.

4.4 Error Handling, Logging and Monitoring

- All API errors return a consistent JSON envelope: {"error_code", "message", "details"} with appropriate HTTP status codes (400/401/403/404/409/500).
- Validation errors (e.g., duplicate attendance marking) are raised as 409 Conflict with a descriptive error_code so the frontend can show a precise message.
- Server-side exceptions are logged centrally (Python logging module → rotating file handler in development, structured JSON logs in production) with request correlation IDs.
- Critical failures (e.g., failed email notification on correction request) are retried once and logged as warnings rather than failing the parent request.
- CI pipeline runs automated tests on every pull request; failures block merge (see Test Plan, Section 5).

4.5 UX Design

- Student dashboard: single-page summary of attendance percentage per course, with a shortage warning banner when below the configured threshold.

- Faculty attendance-marking screen: roster-style checklist with bulk "mark all present" and per-student toggle, optimized for marking a full section in under two minutes.
- Correction-request flow: students submit a reason via a short form; faculty see a single pending-requests queue with one-click approve/reject.
- Consistent Bootstrap 5 component library across all three role-specific UIs for visual consistency and reduced frontend effort (owned by the UI/UX & frontend support secondary responsibility).

4.6 Open Issues and Next Steps

- Confirm production hosting target (e.g., free-tier PaaS) before the final deployment sprint.
- Decide the exact length of the attendance-record edit window (FR-038) in hours or days, and whether it is configurable per course like the shortage threshold (FR-035).
- Decide whether JWT refresh tokens are needed for this semester's scope, or whether a single short-lived access token is sufficient given the system's academic (non-continuous) usage pattern.
- Finalize the SonarQube (or equivalent) quality-gate thresholds to enforce in the CI pipeline before Sprint 2.

5. Appendices

5.1 Glossary

Term	Definition
Attendance Session	A single faculty-initiated instance of attendance-taking for one course-section on one date/period.
Defaulter	A student whose attendance percentage falls below the configured shortage threshold.
Correction Request	A student-raised request to amend an attendance record, subject to faculty approval.

5.2 References

- ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description.
- OWASP Top 10 (2021) — Web Application Security Risks.
- NIST SP 800-160 — Systems Security Engineering.
- AttendEase Software Requirements Specification (SRS), v1.2, Team 3.

5.3 Tools

- Graphviz — component/architecture diagram generation.
- Matplotlib (Python) — UML sequence diagram generation.
- draw.io / Swagger (planned) — ongoing diagram and API-contract maintenance during implementation.